UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACION DE ARCHIVOS

PROYECTO 2 (Reporte Final)

NOMBRE:	**CARNET:**
Marvin Antonio Vásquez Chán	1558525

QUETZALTENANGO, NOVIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S. J

1. Introducción

El presente documento describe el desarrollo e implementación del Laboratorio No. 2 de Manejo e Implementación de Archivos, cuyo propósito es construir una aplicación de escritorio que administre una agenda de contactos mediante archivos de registros de tamaño fijo.

El sistema almacena contactos, los localiza por medio de identificadores únicos y modifica registros específicos sin recorrer ni reescribir el archivo completo. Para lograrlo se implementó acceso aleatorio con las funciones seek() y tell(), acompañado de un índice primario en formato JSON.

Además, la aplicación exporta la información a JSON y PDF, verifica la integridad de los datos con SHA-256, crea respaldos comprimidos en ZIP y consulta metadatos y permisos de los archivos. Todo se desarrolló en Python con PySide6, con una estructura modular del código.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio para administrar una agenda de contactos mediante archivos de registros de tamaño fijo, implementando acceso aleatorio, indexación, exportación, verificación de integridad y generación de respaldos.

2.2 Objetivos específicos

- Almacenar contactos en registros de tamaño fijo.
- Utilizar seek() y tell() para acceder directamente a la posición de cada registro.
- Crear un índice primario en formato JSON para buscar por identificador.
- Agregar, buscar y modificar contactos.
- Generar exportaciones JSON y reportes PDF.
- Verificar la integridad del archivo de datos mediante SHA-256.
- Crear respaldos comprimidos en ZIP y comparar tamaños.
- Consultar los metadatos y permisos de los archivos almacenados.

3. Descripción general del sistema

Cada contacto tiene cinco campos: identificador, nombre, teléfono, correo electrónico y ciudad. Los datos se guardan en contactos.dat, donde todos los registros ocupan exactamente la misma longitud. El archivo indice.json relaciona cada identificador con el número de registro que ocupa en contactos.dat, y el archivo contactos.sha256 conserva la huella digital del archivo de datos para detectar modificaciones externas.

3.1 Tecnologías utilizadas

Tecnología	Aplicación dentro del proyecto

Python 3.14	Lenguaje principal de programación
PySide6	Interfaz gráfica de escritorio
JSON (json)	Índice primario y exportación de contactos
ReportLab	Generación del reporte PDF
hashlib	Verificación de integridad SHA-256
zipfile	Creación de respaldos comprimidos
os y pathlib	Rutas, metadatos y permisos
Git y GitHub	Control de versiones
PyCharm	Entorno de desarrollo

3.2 Estructura del proyecto

```
proyecto2_manejo_implementacion_archivos/
├── main.py
├── requirements.txt
├── README.md
├── data/           contactos.dat · indice.json · contactos.sha256
├── exports/        contactos.json · reporte_contactos.pdf
├── backups/        respaldo_contactos.zip
└── src/agenda_contactos/
    ├── almacenamiento/   registros.py · indice.py · integridad.py
    ├── contactos/        modelo.py · servicio.py
    ├── exportaciones/    json_exportador.py · pdf_exportador.py
    ├── herramientas/     respaldos.py · metadatos.py
    └── interfaz/         ventana_principal · ventana_contactos
                          ventana_exportar · ventana_herramientas · estilos
```

El código se separa en capas: almacenamiento (archivos de bajo nivel), contactos (modelo y reglas de negocio), exportaciones, herramientas e interfaz. La interfaz nunca toca los archivos directamente; siempre pasa por ServicioContactos.

4. Estructura y organización de los registros

Todos los registros deben tener la misma longitud sin importar cuántos caracteres contenga cada campo. Para lograrlo se definió una longitud máxima por campo y se rellenó con espacios lo que no alcanza esa longitud.

4.1 Distribución de los campos

Campo	Longitud (bytes)	Desplazamiento dentro del registro
ID	6	0 – 5
Nombre	40	6 – 45
Teléfono	15	46 – 60
Correo electrónico	45	61 – 105

Ciudad	30	106 – 135
Total	**136**	

4.2 Cálculo del tamaño total

Tamaño del registro = 6 + 40 + 15 + 45 + 30 = 136 bytes

En el código, el tamaño se calcula a partir del diccionario de longitudes, por lo que cambiar un campo actualiza automáticamente todas las posiciones:

```
LONGITUDES = {"id": 6, "nombre": 40, "telefono": 15,
              "correo": 45, "ciudad": 30}
TAMANO_REGISTRO = sum(LONGITUDES.values())   # 136
```

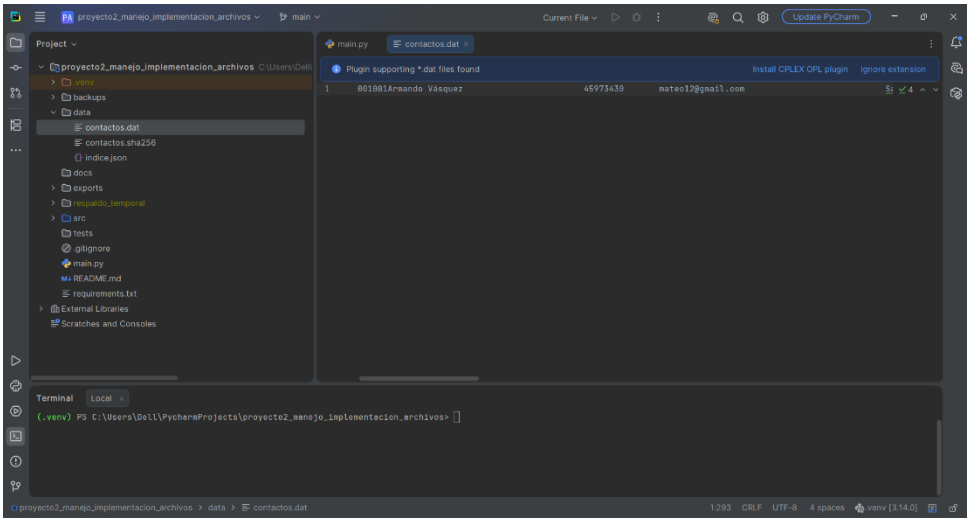
4.3 Longitudes medidas en bytes (UTF-8)

Las longitudes se aplican en bytes y no en caracteres, porque el archivo se codifica en UTF-8 y las letras con tilde ocupan 2 bytes. Por ejemplo, "Armando Vásquez" tiene 15 caracteres pero 16 bytes. La función ajustar_campo() recorta el texto sin partir un carácter a la mitad y luego rellena con espacios hasta la longitud exacta del campo. El ID se guarda con ceros a la izquierda (1001 → 001001).

4.4 Registros reales almacenados

El archivo contactos.dat contiene actualmente 4 registros y pesa 544 bytes (4 × 136), lo que confirma que no hay registros incompletos.

N.º de registro	Posición (bytes)	ID	Nombre
0	0	1001	Armando Vásquez
1	136	1002	Fernando Perez
2	272	1003	Melqui García
3	408	1004	Daniel Hardman



5. Implementación del acceso aleatorio

El acceso aleatorio permite llegar a un registro específico sin leer los anteriores. Se utilizan seek() y tell() de Python con el archivo abierto en modo binario.

5.1 Función seek()

seek() mueve el cursor del archivo a una posición determinada. En el proyecto la posición se calcula con:

Posición = Número de registro × Tamaño del registro

Por ejemplo, el registro número 3 (el cuarto, porque se cuenta desde 0) está en 3 × 136 = 408, y corresponde a Daniel Hardman (ID 1004).

5.2 Función tell()

tell() devuelve la posición actual del cursor. Se utiliza en tres momentos: al agregar (para saber en qué byte empieza y termina el nuevo registro), al leer (para confirmar que seek() aterrizó donde se esperaba) y al modificar (para validar la ubicación antes de escribir). La interfaz muestra estas posiciones al usuario.

5.3 Código real del proyecto

Lectura de un registro (registros.py):

```
def leer_registro(numero_registro):
    with open(RUTA_CONTACTOS, "rb") as archivo:
        posicion = numero_registro * TAMANO_REGISTRO
        archivo.seek(posicion)
        posicion_actual = archivo.tell()
        datos = archivo.read(TAMANO_REGISTRO)
        if len(datos) != TAMANO_REGISTRO:
            raise ValueError("No se encontró un registro completo.")
        contacto = decodificar_registro(datos)
    return contacto, posicion_actual
```

Agregar al final del archivo (registros.py):

```
with open(RUTA_CONTACTOS, "a+b") as archivo:
    archivo.seek(0, 2)          # final del archivo
    posicion = archivo.tell()   # byte donde empezará el registro
    if posicion % TAMANO_REGISTRO != 0:
        raise ValueError("El archivo contiene registros incompletos.")
    numero_registro = posicion // TAMANO_REGISTRO
    archivo.write(crear_registro(contacto))
```

5.4 Modificación de registros

Al modificar un contacto, el sistema consulta primero el índice para obtener su número de registro, abre el archivo en modo r+b (lectura y escritura sin borrar el contenido), se ubica con seek() y sobrescribe únicamente esos 136 bytes. Como todos los registros miden lo mismo, el nuevo contenido siempre cabe exactamente en el espacio del anterior y los demás contactos quedan intactos.

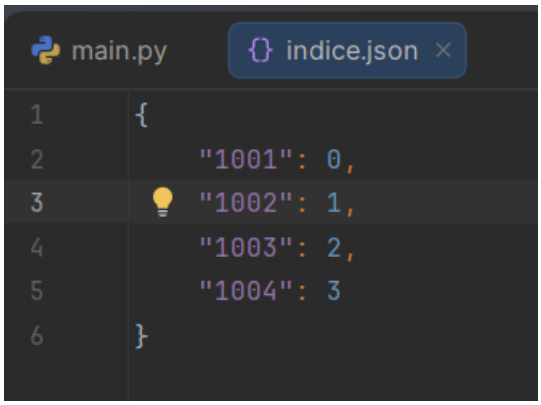
```
with open(RUTA_CONTACTOS, "r+b") as archivo:
    posicion = numero_registro * TAMANO_REGISTRO
    archivo.seek(posicion)
    if archivo.tell() != posicion:
        raise ValueError("No se pudo ubicar el registro.")
    archivo.write(crear_registro(contacto))
```

Prueba realizada: al modificar el contacto 1002 sobre una copia de prueba, el tamaño del archivo no cambió y los únicos bytes alterados estaban dentro del rango 136–271 (registro 1).

6. Índice primario en JSON

Para no recorrer todo el archivo en cada búsqueda se mantiene indice.json, que relaciona el identificador de cada contacto con su número de registro. Contenido real:

```
{
    "1001": 0,
    "1002": 1,
    "1003": 2,
    "1004": 3
}
```



6.1 Funcionamiento de la búsqueda

1. El usuario escribe el ID y presiona Buscar por ID.
2. El sistema carga y valida indice.json.
3. Se obtiene el número de registro asociado al ID.

4. Se calcula la posición: registro × 136.
5. Se usa seek() para ir directamente a ese byte.
6. Se lee el registro, se decodifica y se verifica que el ID leído coincida con el buscado.
7. Los datos se muestran en el formulario junto con el registro y la posición.

6.2 Mantenimiento y seguridad del índice

- Al agregar un contacto se rechaza cualquier ID que ya exista en el índice.
- El índice se guarda de forma segura: primero se escribe en un archivo temporal (indice.tmp), se fuerza la escritura a disco con fsync() y luego se reemplaza el original con os.replace(). Así, si ocurre un fallo a mitad de la operación, el índice anterior no se corrompe.
- Al cargarlo se valida que sea un diccionario, que las claves sean numéricas y que las posiciones sean enteros no negativos.
- Si el índice y el registro no coinciden, se lanza un error en lugar de mostrar datos incorrectos.

6.3 Comparación de métodos de búsqueda

Método	Funcionamiento	Complejidad
Búsqueda secuencial	Lee registros uno por uno hasta encontrar el ID	O(n)
Consulta del índice	Busca el ID en un diccionario (una vez cargado el JSON)	O(1) promedio
Acceso al registro	Calcula la posición y usa seek()	O(1)

Con el índice, el costo de leer el archivo de datos pasa a ser constante: siempre se lee un solo registro de 136 bytes, sin importar si hay 4 o 100 000 contactos.

Nota de alcance: en la implementación actual indice.json se vuelve a cargar desde disco en cada consulta. La búsqueda en el diccionario es O(1), pero la carga del JSON crece con la cantidad de contactos. Una mejora futura sería mantener el índice en memoria durante la ejecución y escribirlo a disco solo cuando cambie.

7. Verificación de integridad con SHA-256

Se utilizó el módulo hashlib para calcular una huella digital de contactos.dat. Esta huella se almacena en contactos.sha256 y permite detectar modificaciones hechas fuera de la aplicación.

7.1 Funcionamiento

- El archivo se lee en bloques de 4096 bytes para que el consumo de memoria sea constante, incluso con archivos grandes.
- Cada vez que se agrega o modifica un contacto desde la aplicación, el hash se recalcula y se guarda automáticamente.

- Al iniciar el programa se verifica la integridad y, si hay diferencia, se muestra la ventana "Advertencia de integridad".
- Desde Herramientas → Verificar integridad SHA-256 se puede comprobar manualmente en cualquier momento.
- Si no existe contactos.sha256, se crea el hash inicial y se informa al usuario.

```
def calcular_hash():
    sha256 = hashlib.sha256()
    with open(RUTA_CONTACTOS, "rb") as archivo:
        while True:
            bloque = archivo.read(4096)
            if not bloque:
                break
            sha256.update(bloque)
    return sha256.hexdigest()
```

7.2 Estados de la verificación

Estado	Significado	Mensaje en la aplicación
correcto	El hash actual coincide con el guardado	"El archivo contactos.dat conserva su integridad."
alterado	El hash actual es distinto del guardado	"ADVERTENCIA: el archivo contactos.dat fue modificado o presenta diferencias respecto al hash guardado."
nuevo	No existía hash; se creó uno	"Se creó el registro inicial de integridad SHA-256."
sin_archivo	Aún no existe contactos.dat	"Todavía no existe el archivo de contactos."

7.3 Prueba de detección de modificaciones

El hash vigente de contactos.dat es:

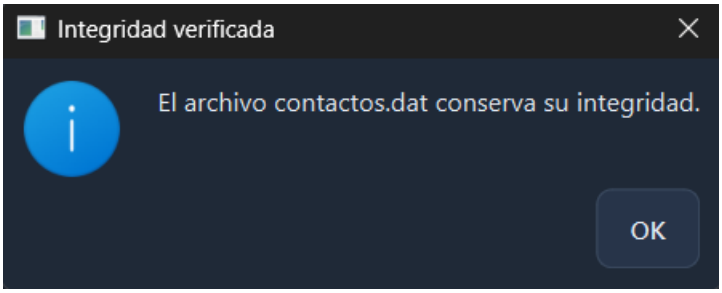
1dd8e48d757fcba6ce39f870d8998dfa8a90a87acafa7764cd01302fc35cdff7

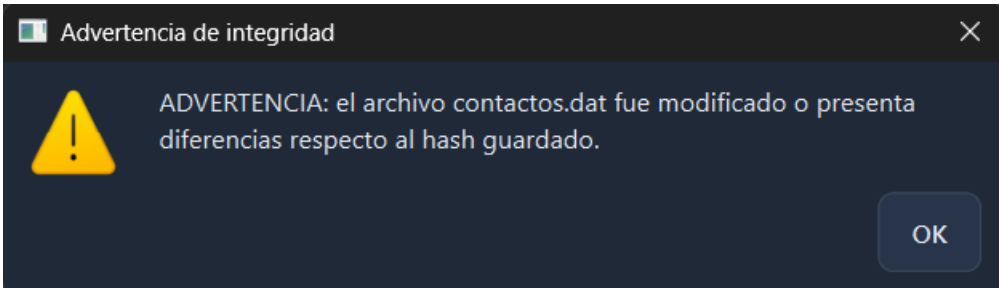
Para la prueba se conservó una copia de recuperación en la carpeta respaldo_temporal/ (contactos_original.dat y hash_original.sha256), con el mismo hash anterior, y se excluyó del repositorio mediante .gitignore.

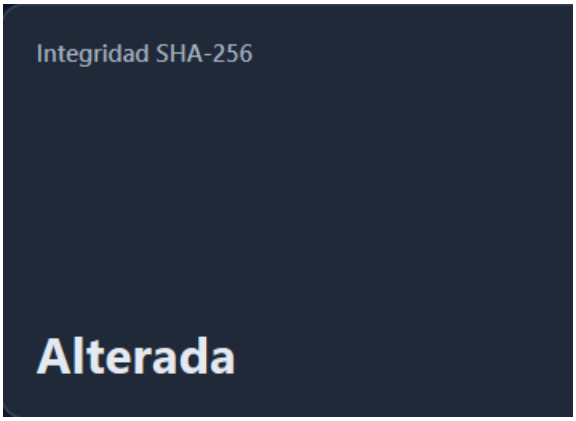
Procedimiento:

1. Verificar la integridad con el archivo original: resultado correcto.
2. Modificar manualmente unos pocos bytes de contactos.dat (por ejemplo, con un editor hexadecimal o de texto).
3. Volver a verificar desde la aplicación: resultado alterado.
4. Restaurar contactos_original.dat en data/ y comprobar que vuelve a aparecer "correcto".

Adicionalmente, esta misma secuencia se reprodujo mediante un script sobre una copia del proyecto: con el archivo intacto el estado fue correcto; tras sobrescribir 2 bytes en la posición 10, el estado fue alterado.







Limitación conocida: como el hash se actualiza automáticamente tras cada alta o modificación hecha en la aplicación, una alteración externa que no haya sido detectada antes de operar quedaría registrada como válida. Por eso el sistema verifica al iniciar, antes de permitir operaciones. Una mejora sería verificar también inmediatamente antes de cada escritura.

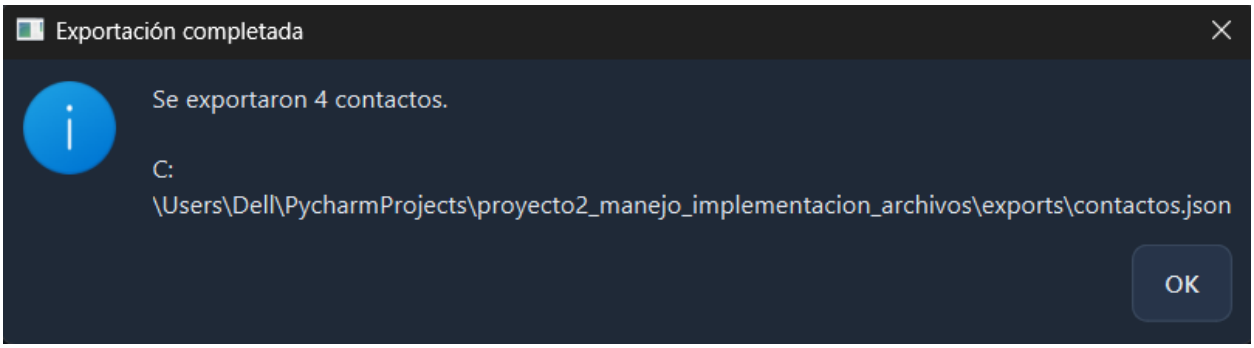
8. Exportación de información y reportes

8.1 Exportación JSON

La opción Exportar → Exportar contactos a JSON lee los registros mediante el índice, los ordena por ID y genera exports/contactos.json con codificación UTF-8 (sin escapar tildes) y sangría de 4

espacios. Cada contacto conserva sus cinco campos: id, nombre, teléfono, correo y ciudad. La aplicación muestra el mensaje "Se exportaron 4 contactos" junto con la ruta del archivo.


```
{
  "id": 1001,
  "nombre": "Armando Vásquez",
  "telefono": "45973430",
  "correo": "mateo12@gmail.com",
  "ciudad": "Salcajá"
}
```

8.2 Reporte PDF

La opción Generar reporte PDF primero actualiza el JSON y luego construye el PDF a partir de él con ReportLab. El documento se genera en hoja carta horizontal e incluye el título "REPORTE DE CONTACTOS", la fecha y hora de generación, una tabla con ID, nombre, teléfono, correo y ciudad (con encabezado azul y filas alternadas) y el total de contactos registrados. El encabezado de la tabla se repite si el reporte ocupa más de una página.



REPORTE DE CONTACTOS

Fecha de generación: 09/11/2026 00:34:10

ID	Nombre	Teléfono	Correo	Ciudad
1001	Armando Vásquez	45973430	mateo12@gmail.com	Salcajá
1002	Fernando Perez	55151625	yordy09@gmail.com	Huehuetenango
1003	Melqui García	45625131	melqui67@gmail.com	Retahuleu
1004	Daniel Hardman	54120454	hardmand@gmail.com	Guatemala

Total de contactos registrados: 4

9. Respaldos comprimidos en ZIP

La herramienta Crear respaldo ZIP usa el módulo zipfile con compresión DEFLATE y guarda en backups/respaldo_contactos.zip una copia de contactos.dat e indice.json. Los archivos se guardan

con su nombre simple, sin rutas internas, y si falta alguno de los dos la operación se cancela con un mensaje claro.

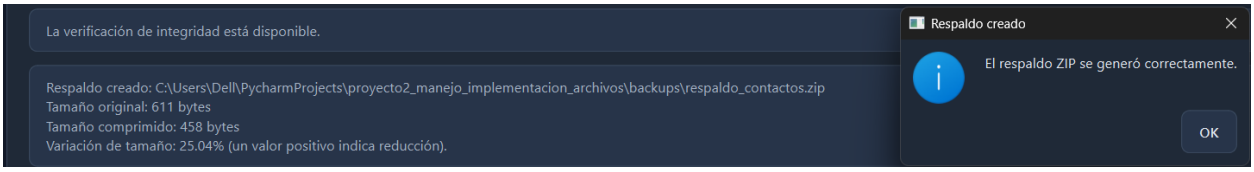
9.1 Comparación de tamaños

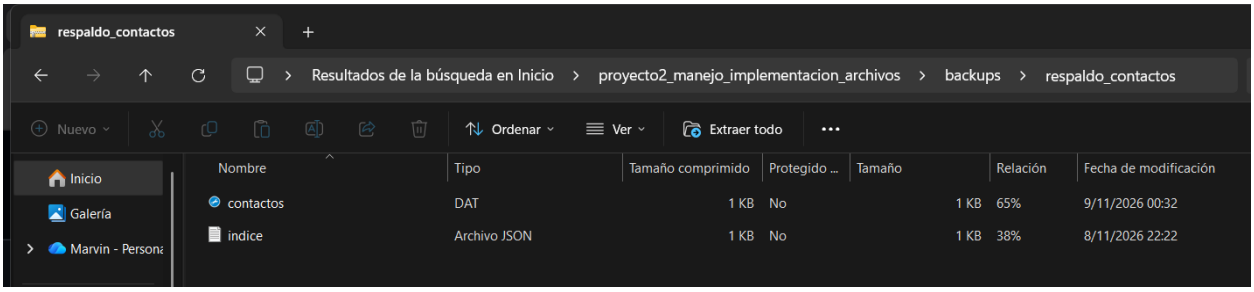
Los valores corresponden al respaldo real almacenado en el proyecto:

Descripción	Tamaño
contactos.dat	544 bytes
indice.json	67 bytes
Tamaño original (suma)	611 bytes
Respaldo ZIP (archivo en disco)	458 bytes
Diferencia	153 bytes
Reducción	25.04 %

Diferencia = 611 − 458 = 153 bytes; Reducción = (1 − 458 / 611) × 100 = 25.04 %.

Dentro del ZIP, contactos.dat pasa de 544 a 194 bytes (64 % de compresión) e indice.json de 67 a 42 bytes (37 %). El archivo ZIP completo pesa más que esos datos comprimidos (236 bytes) porque incluye su propia estructura interna (encabezados y directorio central). Con archivos tan pequeños ese costo fijo es notable; con una agenda de miles de contactos la proporción de ahorro sería mayor, ya que el relleno con espacios es muy compresible.





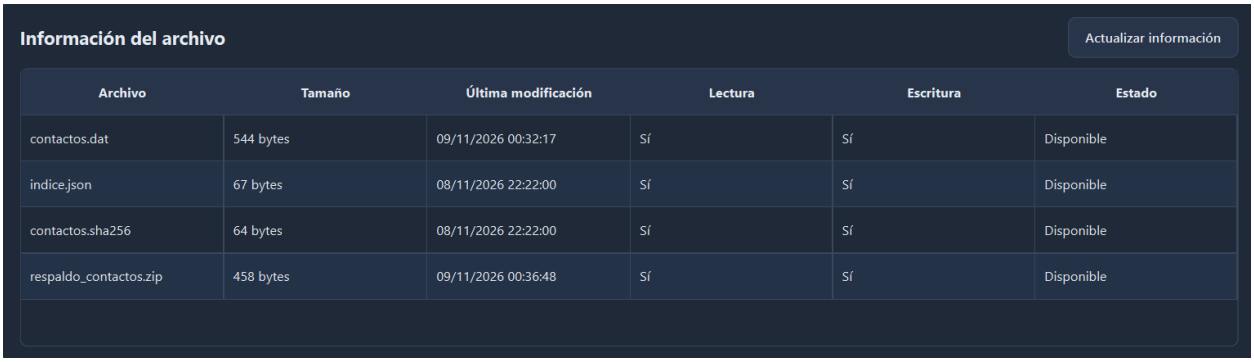
10. Metadatos y permisos de los archivos

La sección Herramientas → Información del archivo consulta, con os.stat() y os.access(), los metadatos de cuatro archivos: contactos.dat, indice.json, contactos.sha256 y respaldo_contactos.zip. La tabla muestra:

- Tamaño en bytes.

- Fecha y hora de última modificación.
- Permiso de lectura (Sí/No).
- Permiso de escritura (Sí/No).
- Estado (Disponible / No existe / mensaje de error).

Si un archivo no existe se informa "No existe" en lugar de producir un error. Con los datos actuales, los tamaños esperados son 544 bytes para contactos.dat, 67 bytes para indice.json, 64 bytes para contactos.sha256 (los 64 caracteres hexadecimales del hash) y 458 bytes para el ZIP.

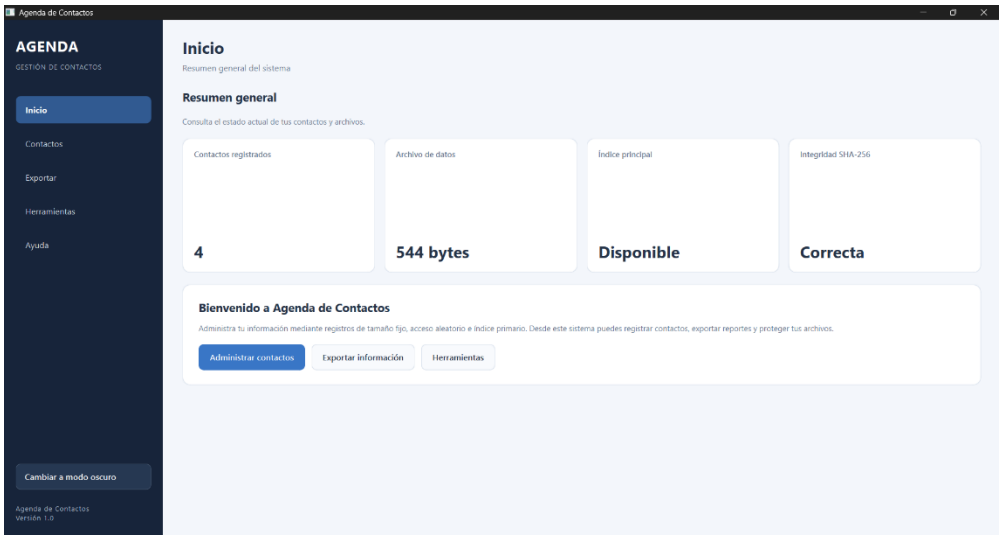


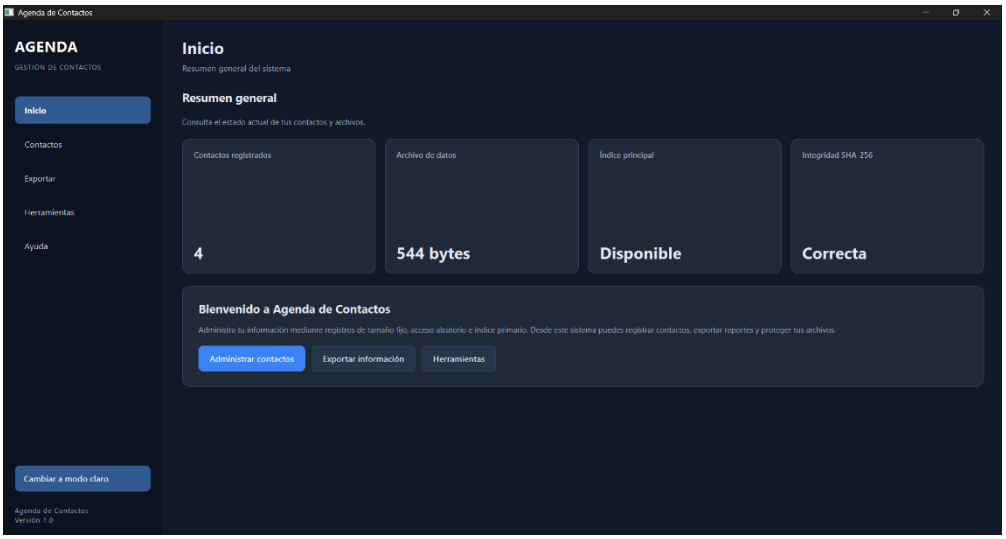
11. Interfaz gráfica y funcionamiento general

La aplicación se construyó con PySide6 en una sola ventana con menú lateral (Inicio, Contactos, Exportar, Herramientas y Ayuda) y soporte de modo claro y oscuro; la preferencia de tema se conserva entre sesiones con QSettings.

11.1 Inicio

Muestra cuatro tarjetas de resumen: cantidad de contactos registrados, tamaño del archivo de datos, estado del índice y estado de la integridad SHA-256, además de accesos directos a las demás secciones.





11.2 Administración de contactos

El formulario permite registrar, buscar y modificar contactos, y la tabla inferior lista todos los registros; al seleccionar una fila sus datos se cargan en el formulario.

Botón	Acción
Registrar contacto	Valida y agrega un registro al final del archivo
Buscar por ID	Localiza el registro usando el índice y seek()
Guardar cambios	Sobrescribe el registro existente
Limpiar formulario	Vacía los campos

Validaciones implementadas:

- El ID debe ser un entero entre 1 y 999 999.
- Nombre, teléfono, correo y ciudad son obligatorios.
- Ningún campo puede exceder su longitud en bytes.
- No se permiten IDs duplicados.
- El ID de un contacto existente no se puede cambiar.

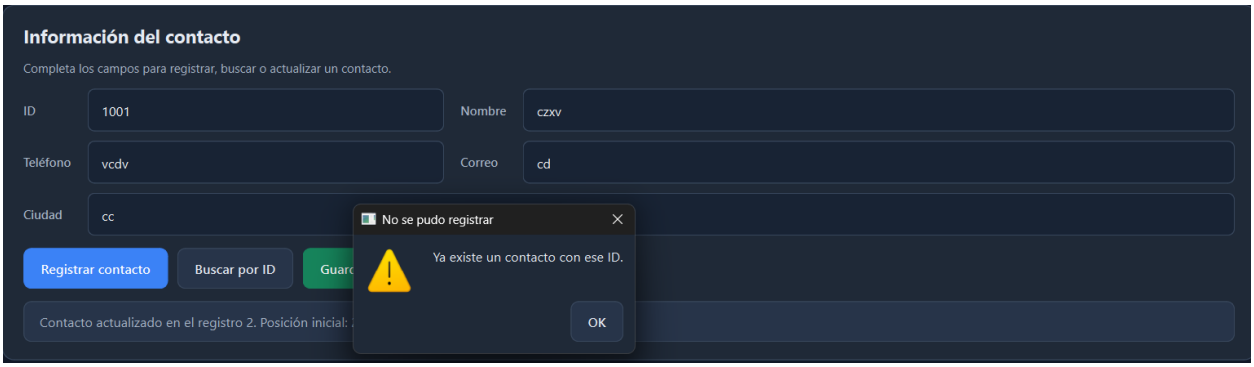
Tras cada operación, la barra de estado informa el número de registro y las posiciones inicial y final en bytes (por ejemplo, "Registro: 4 | Posición inicial: 544 bytes | Posición final: 680 bytes").




Contacto registrado. Registro: 4 | Posición inicial: 544 bytes | Posición final: 680 bytes.
ontactos registrados
ID
Nombre
Teléfono
Correo electrónico
Ciudad
1001
Armando Vásquez
45973430
mateo12@gmail.com
Salcajá
1002
Fernando Perez
55151625
yordy09@gmail.com
Huehuetenango
1003
Melqui García
45625131
melqui67@gmail.com
Retahuleu
1004
Daniel Hardman
54120454
hardmand@gmail.com
Guatemala
1005
Diego Barrios
45141589
diegobarr@gmail.com
Totonicapan

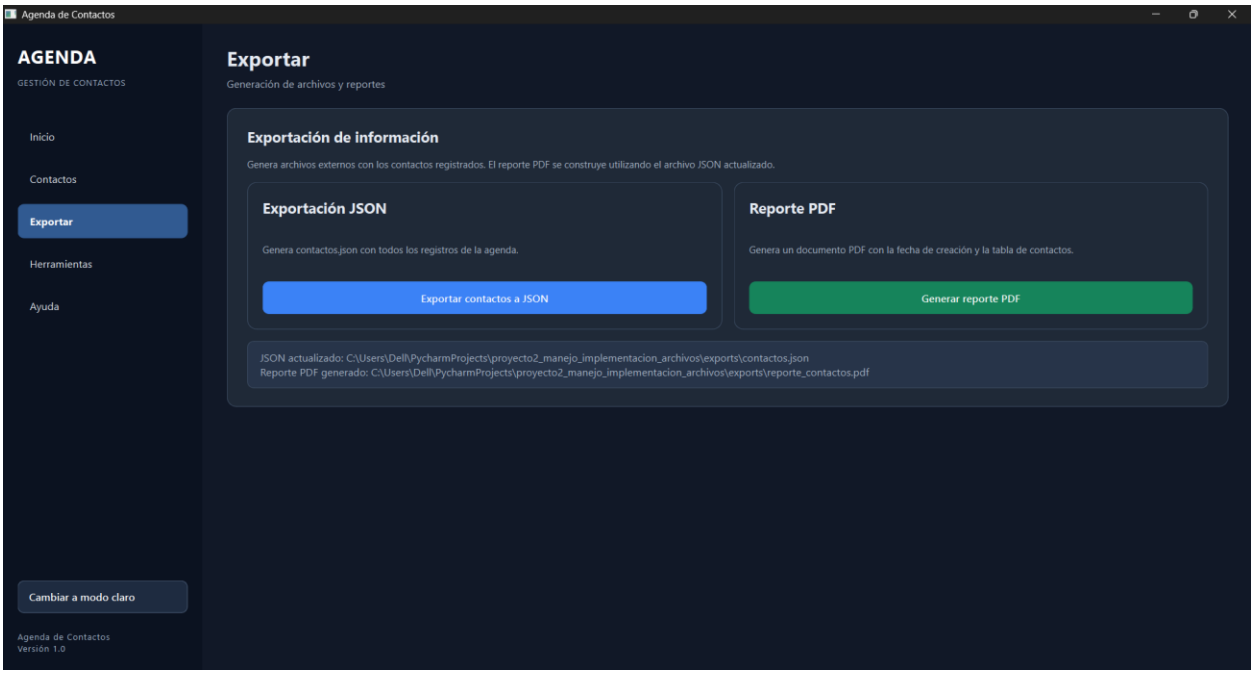

Información del contacto
Completa los campos para registrar, buscar o actualizar un contacto.
ID
1003
Nombre
Melqui García
Teléfono
45625131
Correo
melqui67@gmail.com
Ciudad
Retahuleu
Registrar contacto
Buscar por ID
Guardar cambios
Limpiar formulario
Contacto encontrado en el registro 2, posición 272 del archivo.

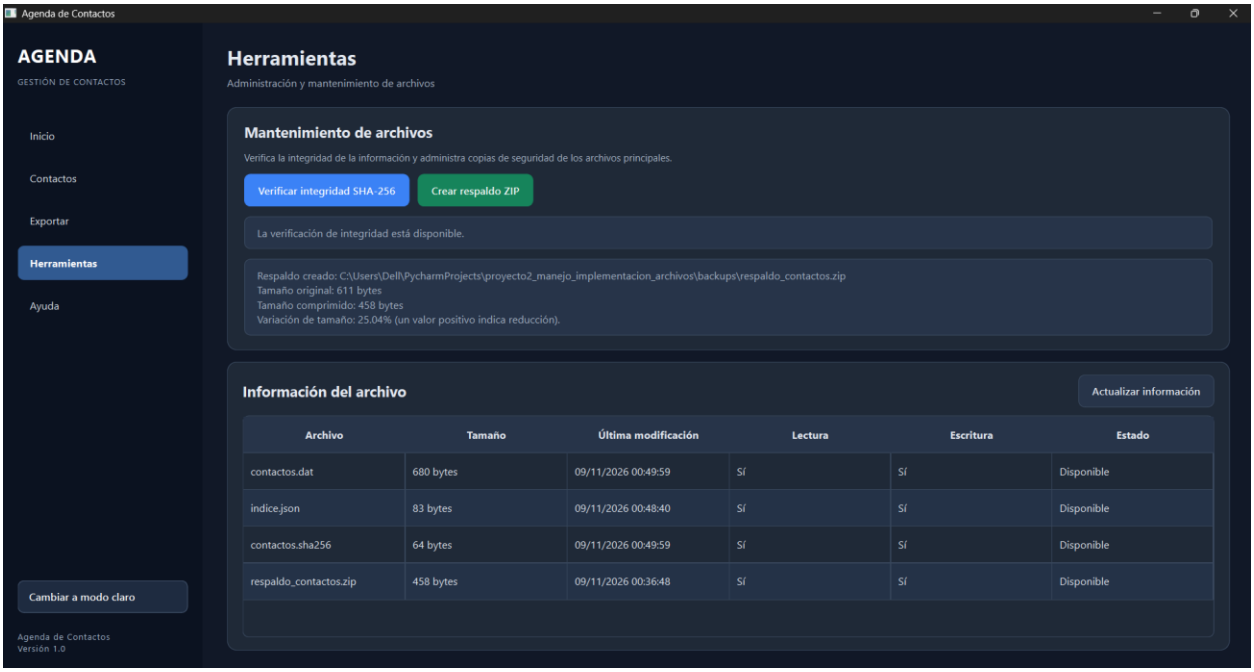

Información del contacto
Completa los campos para registrar, buscar o actualizar un contacto.
ID
Ej. 1001
Nombre
Nombre completo
Teléfono
Número telefónico
Correo
Correo electrónico
Ciudad
Ciudad
Registrar contacto
Buscar por ID
Guardar cambios
Limpiar formulario
Contacto actualizado en el registro 2. Posición inicial: 272 bytes | Posición final: 408 bytes.


Información del contacto
Completa los campos para registrar, buscar o actualizar un contacto.
ID
1001
Nombre
czxv
Teléfono
vcdv
Correo
cd
Ciudad
cc
Registrar contacto
Buscar por ID
No se pudo registrar
Ya existe un contacto con ese ID.
OK

11.3 Exportar y Herramientas

La sección Exportar contiene las dos tarjetas de exportación (JSON y PDF). La sección Herramientas agrupa la verificación de integridad, el respaldo ZIP y la tabla de metadatos. Ambas se describieron en los apartados 7 a 10.





12. Pruebas realizadas

Las siguientes pruebas se ejecutaron sobre una copia del proyecto para no alterar los datos originales:

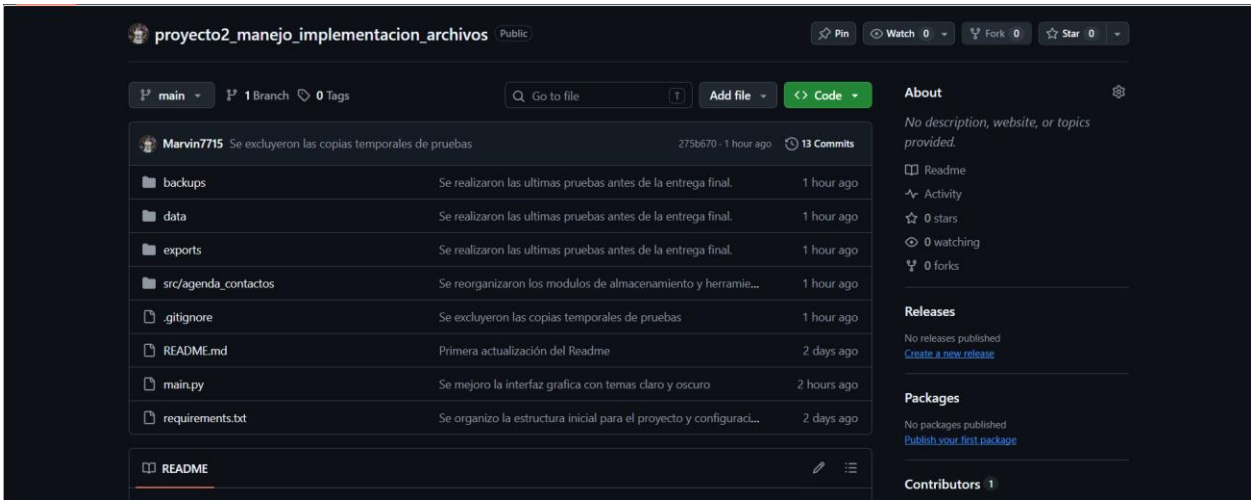
N.º	Prueba	Resultado
1	Verificar integridad con archivo original	correcto
2	Buscar ID 1003	Registro 2, posición 272 (2 × 136), Melqui García
3	Agregar ID 1005	Registro 4, posición inicial 544, final 680; archivo de 680 bytes
4	Modificar ID 1002	Solo cambia el registro 1; tamaño del archivo sin variación
5	Agregar ID repetido (1001)	Rechazado: "Ya existe un contacto con ese ID."
6	Nombre de 41 bytes	Rechazado: "El campo nombre supera su límite de 40 bytes."
7	Alterar 2 bytes de contactos.dat	Integridad: alterado
8	Crear respaldo ZIP	Se genera con contactos.dat e indice.json
9	Exportar JSON	Se genera contactos.json correctamente

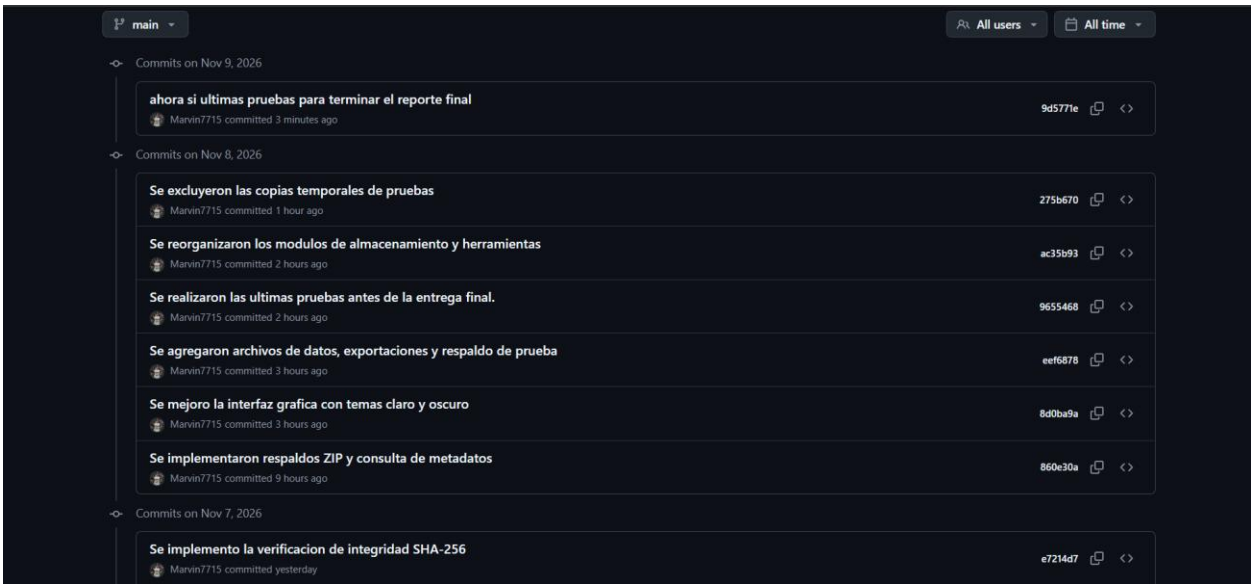
13. Control de versiones y repositorio Git

El proyecto se versionó con Git y se publicó en GitHub, con 13 commits incrementales entre el 6 y el 8 de noviembre de 2026. La rama local main está sincronizada con origin/main.

Repositorio: https://github.com/Marvin7715/proyecto2_manejo_implementacion_archivos

Fecha	Commit	Descripción
06/11 22:01	70f5e2b	Initial commit
07/11 00:24	bb1f90c	Primera actualización del README
07/11 00:27	33c1c20	Estructura inicial del proyecto y configuración de PySide6
07/11 00:49	98f7b29	Primer boceto de la interfaz principal y navegación
07/11 12:01	37f0d85	Almacenamiento de registros e índice en JSON
07/11 13:07	16906cc	Exportación de contactos a JSON y PDF
07/11 21:40	e7214d7	Verificación de integridad SHA-256
08/11 15:49	860e30a	Respaldos ZIP y consulta de metadatos
08/11 21:43	8d0ba9a	Mejora de la interfaz con temas claro y oscuro
08/11 21:51	eef6878	Archivos de datos, exportaciones y respaldo de prueba
08/11 22:27	9655468	Últimas pruebas antes de la entrega final
08/11 23:03	ac35b93	Reorganización de módulos de almacenamiento y herramientas
08/11 23:05	275b670	Exclusión de copias temporales de pruebas





14. Resultados obtenidos

- Contactos organizados en registros de tamaño fijo de 136 bytes.
- Acceso aleatorio con seek() y tell() para leer, agregar y modificar registros.
- Índice primario en JSON con guardado seguro (archivo temporal, fsync() y reemplazo atómico).
- Modificación de un registro sin reescribir el archivo completo.
- Exportación a JSON y reporte PDF generado desde el JSON actualizado.
- Verificación de integridad con SHA-256, automática al iniciar y manual desde la interfaz.
- Respaldo ZIP con una reducción de 25.04 % sobre los datos actuales.
- Consulta de tamaño, fecha de modificación y permisos de lectura y escritura.
- Interfaz en PySide6 con modo claro y oscuro, validaciones y mensajes de posición en bytes.
- Código modular publicado en GitHub con historial de commits incrementales.

15. Conclusiones

1. Los registros de tamaño fijo permiten calcular la posición exacta de cualquier contacto con una multiplicación simple (registro × 136), lo que hace posible el acceso directo sin recorrer el archivo.
2. seek() y tell() permitieron comprender cómo se administra el cursor de un archivo y verificar con precisión en qué byte se lee o se escribe cada registro. Trabajar en modo binario y medir los campos en bytes fue clave para que las letras con tilde no rompieran la alineación.
3. El índice primario en JSON evita la búsqueda secuencial: el sistema obtiene el número de registro a partir del ID y lee un único registro. Además, validar que el ID leído coincida con el buscado protege contra índices desactualizados.
4. SHA-256 ofrece una forma sencilla y confiable de detectar modificaciones externas del archivo de datos. Al mismo tiempo, el trabajo mostró que la verificación debe hacerse antes de operar, porque el hash se actualiza con cada cambio legítimo.
5. El respaldo ZIP redujo los 611 bytes originales a 458 bytes (25.04 %). Con archivos tan pequeños la estructura propia del ZIP limita el ahorro, pero la compresión interna de contactos.dat (64 %) evidencia que el relleno con espacios se comprime muy bien.
6. La interfaz en PySide6 integró todas las funciones en una aplicación organizada, y la separación en capas (almacenamiento, servicio, interfaz) facilitó probar y mantener el código.

16. Recomendaciones y trabajo futuro

- Mantener el índice en memoria durante la ejecución y escribirlo a disco solo cuando cambie.
- Verificar el hash inmediatamente antes de cada escritura para no registrar como válida una alteración externa.
- Agregar la eliminación de contactos (por ejemplo, con una marca de registro borrado) y la reconstrucción del índice a partir de contactos.dat.
- Incluir pruebas automatizadas en la carpeta tests/, que actualmente está vacía.
- Incluir contactos.sha256 en el respaldo ZIP.

17. Anexos

Archivos de almacenamiento: contactos.dat, indice.json, contactos.sha256.

Archivos de exportación: contactos.json y reporte_contactos.pdf.

Archivo de respaldo: respaldo_contactos.zip.

Código fuente y control de versiones: código completo del proyecto y repositorio en GitHub (https://github.com/Marvin7715/proyecto2_manejo_implementacion_archivos) con su historial de commits.

Instrucciones de ejecución: instalar las dependencias con pip install -r requirements.txt (PySide6 y reportlab) y ejecutar main.py.